

Teorie — moștenire și polimorfism

Notițele din `App.java`, corectate și completate · Java · Adrian

Fișa asta e de **citit**, nu de rezolvat — exercițiile vin separat. Secțiunea 1 ia exact ce ai notat tu în `App.java` și arată ce e corect, ce e scris pe jumătate și ce e greșit. Restul fișei explică mecanismul din spate.

1. Notițele tale, rând cu rând

Cum ai notat	Cum e corect
A extends B → A sn subclasa, B sn superclasa	Corect (doar tastat greșit: „e”, nu „sn”). Se mai spune și clasă derivată pentru A și clasă de bază pentru B.
A are acces la toate campurile protected sau public ale lui B	Incomplet — și e fix cazul tău . Câmpurile <code>private</code> din B există în obiectul lui A, dar A nu le poate scrie pe nume. În <code>User</code> , <code>id</code> și <code>email</code> sunt <code>private</code> — de asta <code>Student.descriere()</code> nu le poate lipi direct și cheamă <code>super.descriere()</code> . Se ajunge la ele prin getter/setter public.
A este obligata la instantiere sa apeleze constructorul lui B	Corect , dar precizează: apelul e prima instrucțiune din constructorul lui A. Dacă nu scrii tu <code>super(...)</code> , Java pune automat <code>super()</code> fără parametri — și dacă B n-are un constructor fără parametri, nu compilează. <code>User</code> n-are, deci <code>super(...)</code> e obligatoriu.
super ne ajuta sa accesam campurile superclasei	Doar jumătate . <code>super</code> are două utilizări diferite: <code>super(...)</code> = apel de constructor (numai pe prima linie), iar <code>super.ceva</code> = acces la câmpul sau metoda din superclasă. A doua e cea importantă la suprascriere: <code>super.descriere()</code> rulează versiunea din <code>User</code> , nu pe a ta.
B poate fi referinta pentru orice subclasa	Corect . De adăugat regula care contează: tipul referinței decide ce ai voie să chemi, obiectul decide ce versiune rulează.
B nu are acces la metodele din subclasa	Corect . Mai exact: printr-o referință <code>User</code> vezi doar ce e declarat în <code>User</code> . Ca să ajungi la <code>getNrCredite()</code> îți trebuie downcasting, cu <code>instanceof</code> înainte.
A x = new A(); B y = x; A a = (A) y;	Corect . Upcasting-ul (<code>B y = x</code>) e implicit și mereu sigur. Downcasting-ul e explicit și poate crăpa la rulare cu <code>ClassCastException</code> . Și: dacă <code>A extends B</code> și <code>C extends B</code> , din A în C nu se poate — sunt frați, nu unul sub altul.
Polimorfismul implica automat mostenirea	Adevărat doar pentru un fel de polimorfism . Suprascrierea (override) cere moștenire — da. Dar supraîncărcarea (overload: același nume, parametri diferiți, ca cei doi constructori ai tăi din <code>User</code>) e tot polimorfism și nu cere moștenire deloc.
early binding = la compilare late binding = la rulare	Corect . Completarea: în Java metodele de instanță sunt implicit late binding. Early binding primesc doar <code>static</code> , <code>private</code> , <code>final</code> , constructorii, alegerea dintre variantele supraîncărcate — și câmpurile .

2. Ce se moștenește, de fapt

Obiectul are tot, tu vezi doar o parte

Când scrii `new Student(...)`, în memorie se face **un singur obiect** care conține și partea de `User` (id, email, password, createdAt), și partea de `Student` (anInscriere, nume, nrCredite). Nu sunt două obiecte legate.

modificator	subclasa îl vede pe nume?
public	da, de oriunde
protected	da, în subclase și în același pachet
(fără nimic)	doar în același pachet
private	nu — deși câmpul există în obiect

Constructorii **nu** se moștenesc. De asta fiecare subclasă își scrie ai ei.

3. Constructorii și `super(...)`

Ordinea reală de execuție

```
Student s = new Student("STUDENT,001,a@b.com,p1,2026-09-09,2024,Ion,20");
```

Se execută de sus în jos: întâi constructorul `User` (pune id, email, password, createdAt), abia apoi corpul constructorului `Student` (pune anInscriere, nume, nrCredite). Partea moștenită e gata înainte să înceapă partea ta.

De aceea în `Student(String text)` ai `super(text)` pe prima linie și apoi mai faci o dată `split` — prima despicare o face `User` pentru câmpurile lui, a doua o faci tu pentru ale tale.

Capcana: `super()` pus automat

```
public class Admin extends User {
    public Admin(String nume) { // fara super(...)
        this.nume = nume;
    }
}
```

```
eroare de compilare:
there is no default constructor available in 'app.users.User'
```

Java a încercat să pună singur `super()`, iar `User` n-are constructor fără parametri. Fix: chemi explicit unul dintre constructorii existenți.

4. Suprascierea (override)

Aceeași semnătură, alt corp

Ca să suprascii, numele + lista de parametri trebuie să fie **identice** cu ale metodei din superclasă. Dacă schimbi ceva, nu suprascii — creezi o metodă nouă, iar polimorfismul nu mai funcționează, fără niciun mesaj de eroare.

```
@Override
public String descriere() {                // suprascie
    return "STUDENT," + super.descriere();
}

public String descriere(int x) {           // NU suprascie: alt parametru
    return "...";
}
```

`@Override` nu schimbă cu nimic execuția — e o verificare la compilare. Dacă scrii din greșală `Descriere()` sau schimbi un parametru, compilatorul te oprește pe loc. Pune-l pe fiecare metodă suprascisă.

`super.metoda()` — reutilizezi, nu copiezi

Fără `super.`, apelul `descriere()` din interiorul lui `Student.descriere()` s-ar chema pe el însuși la infinit (`StackOverflowError`). Cu `super.` ceri explicit versiunea de deasupra.

Atenție: `super.getMarca()` pe o metodă pe care **nu** ai suprascris-o e același lucru cu `getMarca()`. `super.` contează doar când există două versiuni.

5. Upcasting, downcasting, `instanceof`

Referință vs obiect

```
User u = new Student(...);           // upcasting, implicit, mereu sigur

u.descriere();                         // OK: descriere() exista in User
u.getNrCredite();                     // NU compileaza: User nu stie de el

if (u instanceof Student) {           // intrebi INAINTE
    Student s = (Student) u;          // downcasting, explicit
    s.getNrCredite();                  // acum se poate
}
```

Verbal: obiectul e `Student` de la început și rămâne `Student`. Cast-ul nu transformă nimic — doar schimbă „prin ce ochelari” te uiți la el.

Ce se întâmplă fără `instanceof`

```
User u = new Teacher(...);
Student s = (Student) u;           // compileaza! crapa la rulare
```

```
Exception in thread "main" java.lang.ClassCastException:
class app.users.Teacher cannot be cast to class app.users.Student
```

Compilatorul acceptă cast-ul pentru că `Teacher` și `Student` sunt amândouă `User` — teoretic ar fi putut fi un `Student`. Adevărul se află abia la rulare, când se uită la obiectul real. De asta `instanceof` nu e o formalitate.

6. Polimorfism: ce se decide când

Cele două legări

Se decide la compilare (early)	Se decide la rulare (late)
care variantă supraîncărcată se cheamă metode <code>static</code> , <code>private</code> , <code>final</code> constructorii câmpurile (după tipul referinței)	care versiune a unei metode suprascrise rulează — după obiectul real , nu după referință

De asta bucla ta n-are nevoie de `if`

```
for (User u : users) {
    System.out.println(u.descriere());
}
```

Referința e `User` pentru toți, dar la rulare fiecare obiect își cheamă propria `descriere()`. Când adaugi mâine `Admin`, bucla asta rămâne neschimbată — asta e câștigul concret al polimorfismului.

Contrar: varianta cu `if (u instanceof Student) ... else if (u instanceof Teacher)` trebuie modificată la fiecare tip nou. Când te trezești scriind lanțul de `instanceof`, întreabă-te dacă nu cumva îți lipsește o metodă suprascrisă.

Capcana: câmpurile nu sunt polimorfe

```
class User { protected String rol = "USER"; }
class Student extends User { protected String rol = "STUDENT"; }

User u = new Student(...);
System.out.println(u.rol);           // USER
System.out.println(((Student) u).rol); // STUDENT
```

Metodele se aleg după obiect, câmpurile după referință. Cele două câmpuri coexistă în același obiect. Concluzia practică: nu redeclara în subclasă un câmp care există deja în superclasă.

De reținut: referința spune **ce ai voie să chemi**, obiectul spune **ce rulează**. Tot restul (upcasting, `instanceof`, `super`, late binding) sunt consecințe ale acestei propoziții.